ТЕХНИЧЕСКОЕ ЗАДАНИЕ (ТЗ)

Soup Kernel Challenge

---

1. Общие сведения

Заказчик	Soup
Хакатон	Decentrathon 6.0
Название задачи	Soup Kernel Challenge
Тип задачи	ML-инженерия, GPU-программирование (Triton), оптимизация производительности
Формат	команды до 4 человек, один результат на команду
Сроки	30 октября 12:00 – 31 октября 23:59 по времени Астаны (36 часов)
Стартовый репозиторий	github.com/MakazhanAlpamys/soup-kernel-challenge (сейчас закрыт, откроется 30 октября в 12:00)

2. Описание проблемы

Дообучение языковых моделей на одной бесплатной видеокарте упирается в память и время, а не в доступ к модели. Студенты, исследователи и разработчики без дорогих GPU работают на таких картах, как Kaggle T4 (около 15 ГБ, без bf16).

Наши замеры на бесплатной Kaggle T4 (Soup 0.77.0, Qwen2.5-1.5B, 4-бит, batch 4, seq 512, 256 казахских инструкций, 64 шага):

- команда soup train идёт около 80 секунд (из них около минуты само обучение, около 900 токенов/с) и занимает около 14 из 15 ГБ памяти видеокарты;
- Unsloth на тех же данных и той же карте обучает похожую LoRA, занимая около 2,3 ГБ, то есть в 6 раз меньше;
- профиль шага самого Soup на этой задаче: GPU занят около 95% времени; около 36% уходит на умножение матриц, около 44% на мелкие поэлементные операции и приведения типов, около 7% на softmax по словарю из 152 тысяч токенов, около 5% на attention и около 5% на распаковку 4-битных весов.

Значит, запас есть, но неясно, как далеко его можно продвинуть своими кернелами. Это мы и хотим узнать на хакатоне.

Ценность. То же обучение на бесплатной карте идёт быстрее и оставляет запас памяти: на той же T4 можно взять больше данных, длиннее тексты или модель крупнее.

3. Пользователь и сценарий

Основной пользователь — инженер или исследователь без доступа к дорогим видеокартам, который дообучает модель на бесплатной Kaggle T4.

Ключевой сценарий:

1. Пользователь запускает `soup train` с конфигом задачи.
2. Ваши кернелы подключаются к Soup одной функцией apply() (так же делает Unsloth), и тот же запуск идёт быстрее или с меньшим расходом памяти.
3. На выходе тот же адаптер того же качества.

4. Задача хакатона

Напишите свои Triton-кернелы (и любые сопутствующие изменения), которые ускоряют фиксированный запуск обучения Soup на Kaggle T4 или снижают расход памяти при том же качестве.

Какие операции ускорять и как это делать, решаете вы. Обязательно одно: в решении есть хотя бы один ваш собственный Triton-кернел, который реально работает в обучении. Результат оценивается по измеренной скорости и памяти на нашей проверке, потолка у этих шкал нет.

В стартовом репозитории есть всё для запуска и замера: конфиг задачи, загрузка данных, рабочий пример кернела и скрипты измерения. Это те же скрипты, которыми проверяет жюри, поэтому вы видите свой результат до сдачи.

5. Вход и выход

Вход. Soup 0.77.0 (pip install soup-cli[train]==0.77.0) и стартовый репозиторий soup-kernel-challenge с конфигом задачи `task.yaml`:

- модель Qwen2.5-1.5B-Instruct, 4-битные веса (NF4), LoRA r=16, alpha=32 на q_proj и v_proj;
- batch 4, длина до 512 токенов, 256 строк казахских инструкций, одна эпоха (64 шага);
- окружение: бесплатный Kaggle T4 (образ с Python 3.13, torch 2.11, Triton 3.6.0).

При итоговой проверке обучение и оценка идут на скрытых строках того же набора и с другим seed; остальные настройки те же.

Выход. Репозиторий команды, созданный платформой хакатона, с содержимым шаблона soup-kernel-challenge (ссылка на коммит): пакет solution/ с вашими Triton-кернелами и функцией apply(), которая подключает их к Soup. Он запускается в Kaggle-ноутбуке из шаблона командой «выполнить всё» без правок. Мы измеряем полное время команды soup train (от запуска до сохранённого адаптера), пиковую память GPU и качество полученного адаптера.

Пример итога, который печатает run_task.py (числа условные): `speedup 1.300x  memory base/solution 1.800x  eval loss vs base 0.1 %`. Те же числа и подробности по каждому запуску сохраняются в runs/result.json.

6. Данные

Для задачи достаточно одного открытого набора, регистрация и доступ не нужны.

Набор	AmanMussa/kazakh-instruction-v2 (Hugging Face), лицензия MIT: 52 201 казахская инструкция
Поля	`instruction`, `input`, `output` (формат alpaca)

Качество	перевод Stanford Alpaca на казахский через Google Translate с ручной правкой ошибок; добавлены вопросы об истории и культуре Казахстана (автор Mussa Aman, КазНУ). Пустые ответы редки: 2 на 2000 строк
Обучение	первые 256 строк
Открытая выборка для самопроверки	строки 40000–40255
Скрытая выборка	другие строки того же набора, их выбирают организаторы; на них идут обучение и оценка при итоговой проверке

Строка в среднем занимает около 220 токенов (медиана 140), 6% строк длиннее 512 и обрезаются. Для тестов своих кернелов можно применять синтетические тензоры.

7. Must have

Решение считается выполненным, только если проходит все пять пунктов. Пункты 1–4 проверяют скрипты из стартового репозитория, их можно запустить до сдачи.

№	Требование	Как проверяем
1	Решение запускается в чистом Kaggle-ноутбуке на T4 без правок	наш запуск «выполнить всё»
2	Свои Triton-кернелы (`@triton.jit`) из папки solution/ запускаются в каждом замеряемом запуске и вместе занимают не менее 5% времени обучения (считается с 3-го шага: первые два уходят на компиляцию). Не считаются кернелы `torch.compile`, кернелы из примера (как бы их ни переписали), скопированные чужие кернелы и кернелы, чей результат не используется в обучении	замер времени кернелов в самих запусках (CUDA-события), разбор исходников и чтение кода жюри
3	Не хуже базы ни по одному показателю: loss адаптера на скрытых строках выше, чем у базового Soup, не больше чем на 0,5%; время и пик памяти больше, чем у базы, не больше чем на 10%	скрипты eval_adapter.py (одна функция потерь для всех) и run_task.py
4	Та же задача и нет регрессий: те же настройки обучения, адаптер и выходы Soup (число токенов по шагам совпадает; энтропия, точность и потери строк в rewind.jsonl в пределах 3%); кривая потерь всех шагов близка к базе (первый шаг в пределах 1%, в среднем в пределах 3%); замер чистый (одна видеокарта, свободная перед запуском, настройки аллокатора харнесса); тесты Soup (79 тестов в 5 файлах: config, formats, loader, trainer_init, callback) проходят с вашими патчами	скрипт check_rules.py и pytest
5	README: что сделано, как запустить, какие результаты, ссылка на Kaggle-ноутбук	проверка жюри

8. Жёсткие ограничения

Платформа. Бесплатный Kaggle T4 (используется одна карта из двух), fp16, без bf16. Итоговая проверка идёт только на этой платформе.

Доступ. Нужен свой Kaggle-аккаунт с подтверждённым телефоном: без этого Kaggle не даёт GPU и интернет. Бесплатная квота около 30 часов GPU в неделю на аккаунт. Данные открытые, персональных данных в задаче нет.

Разрешено. Пакеты torch, triton, transformers, peft, trl, bitsandbytes и soup-cli 0.77.0 с PyPI; ИИ-агенты; идеи из чужих кернелов с обязательным указанием источника, но не их код.

Нельзя:

- использовать готовые кернел-библиотеки (unsloth, liger-kernel, xformers, flash-attn, torchao и подобные, полный список в harness/common.py), в том числе встроенные режимы Soup backend: unsloth и use_liger;
- выдавать кернелы, сгенерированные `torch.compile`, за свои;
- менять саму задачу: модель, данные, состав и порядок батчей, число шагов, настройки LoRA (в том числе dropout), оптимизатор и расписание lr; переключать встроенные режимы Soup (gradient_checkpointing, activation_offloading, fast_lora, use_cut_ce, packing и подобные); добавлять обучаемые параметры; отключать выходы Soup или менять их значения (метрики в логе, rewind.jsonl). Убирать pad-токены внутри батча своим кодом можно, смешивать строки разных батчей нельзя;
- менять скрипты измерения и оценки (harness/, tests/, task/) и заданные ими настройки, например аллокатор памяти PyTorch; искать или подбирать скрытые данные;
- обходить замер: другая видеокарта, фоновые процессы, особое поведение под замером или в тестах, подмена сохранённого адаптера, готовые скомпилированные кэши; читать скрытые строки и любые файлы вне своего репозитория и данных задачи;
- сдавать более одного результата от команды или командой более 4 человек;
- закрытый код: к сдаче репозиторий и Kaggle-ноутбук должны быть открыты для проверки жюри.

9. Желаемые направления

Это подсказки, а не обязательный список: оценивается измеренный результат, а не выбор направления.

- **Слияние мелких операций.** По нашему профилю шага Soup на этой задаче около 44% времени GPU уходит на поэлементные операции и приведения типов: нормализации и эмбеддинги в fp32, LoRA, активации.
- **Слитая потеря и большой словарь.** Логиты по словарю из 152 тысяч токенов занимают гигабайты памяти и около 7% времени; кроме loss их читают подсчёт энтропии и точности и журнал rewind. Unsloth на тех же данных занимает в 6 раз меньше памяти.
- **Слитое умножение с 4-битными весами.** Распаковка NF4 занимает около 5% времени шага и держит плотную копию весов в памяти; само умножение матриц занимает ещё около 36%.

- **Сжатие активаций, перепаковка весов, int8, удаление pad-токенов внутри батча.**

10. Сдача и артефакты

Всё сдаётся на платформе хакатона. Каждая команда сдаёт четыре вещи:

1. **GitHub-репозиторий команды.** Вся работа должна быть в GitHub-репозитории, который создала для команды платформа хакатона: работу в других репозиториях не проверяем. В нём пакет solution/, README (что сделано, как запустить, результаты, ограничения) и заполненный SUBMISSION.md со ссылками на финальный коммит и на Kaggle-ноутбук. К дедлайну репозиторий должен быть открыт для проверки: публичный или с доступом для жюри через платформу.
2. **Публичный Kaggle-ноутбук** из шаблона templates/kaggle_notebook.ipynb (сохранённая версия с результатами; COMMIT в ней — финальный коммит), который запускается «выполнить всё» без правок. Ссылка на него обязательно должна быть в README и в SUBMISSION.md вашего репозитория.
3. **Демо-видео** до 3 минут: что вы ускорили и что получилось.
4. **Презентация.**

Что отразить в презентации:

- какую операцию вы ускоряли и почему;
- как устроен кернел;
- результаты по скорости, памяти и качеству против базы;
- ограничения решения и дальнейшие шаги.

10.1 Как перенести шаблон в репозиторий от платформы

Кнопка «Use this template» создаёт новый репозиторий и не копирует шаблон в уже созданный. Если репозиторий от платформы пустой или в нём только README, склонируйте его (`git clone`) и в его папке выполните:

```
git fetch https://github.com/MakazhanAlpamys/soup-kernel-challenge.git main
git checkout FETCH_HEAD -- .
git commit -m "Soup Kernel Challenge starter kit"
git push origin HEAD
```

Файлы шаблона заменят одноимённые (например, README.md), остальное в репозитории останется. Без git: скачайте шаблон (Code → Download ZIP), распакуйте и загрузите файлы в свой репозиторий. Если платформа создала репозиторий уже с шаблоном внутри, переносить ничего не нужно.

11. Критерии оценивания

Итог считаем нашим запуском на Kaggle T4: те же скрипты, что в стартовом репозитории, но скрытые строки данных. Базовый Soup и ваше решение запускаем по два раза в одной сессии, в порядке база, решение, решение, база, и для каждой стороны берём среднее двух запусков. Время компиляции кернелов входит в замер.

Критерий	Что оцениваем	Баллы
Работает и правильно	Решение проходит все пять пунктов раздела 7. Всё или ничего: 20 баллов, если пройдены все пункты. Если не пройден хотя бы один, решение не допускается и в рейтинг не попадает.	20
Скорость	Полное время команды `soup train`: от запуска до сохранённого адаптера, вместе с загрузкой модели и компиляцией кернелов. s = время базы / ваше время. Самое большое s среди допущенных команд получает 35 баллов, остальные меньше (как именно — ниже).	35
Память	Пик памяти видеокарты за весь запуск (nvidia-smi, замер каждые 0,1 с). m = пик базы / ваш пик. Самое большое m среди допущенных команд получает 30 баллов, остальные меньше.	30
README и воспроизводимость	Оценивает жюри по трём частям, до 5 баллов за каждую (раздел 11.1).	15
Итого		**100**

11.1 README: из чего складываются 15 баллов

Часть	5 баллов, если	Баллы
Что сделано	понятно, какие операции вы ускорили и почему, как устроены ваши кернелы и откуда взяты идеи (с источниками)	до 5
Как запустить	жюри повторяет ваш результат по README без вопросов к команде: шаги запуска, ссылки на Kaggle-ноутбук и финальный коммит	до 5
Результаты и ограничения	таблица против базы из вашего запуска (время, память, loss) и честно о том, что не получилось и где решение слабее	до 5

Часть, сделанная не полностью, получает от 1 до 4 баллов, отсутствующая — 0.

11.2 Баллы за скорость и память простыми словами

1. Узнайте свои s и m. Их печатает run_task.py в конце ноутбука: `speedup` — это s, `memory base/solution` — это m. Например, база обучается 80 с, ваше решение 64 с: s = 80 / 64 = 1,25.
2. Лучшая команда получает полный балл: самое большое s среди допущенных команд — 35 баллов за скорость, самое большое m — 30 баллов за память. Лучшими по скорости и по памяти могут быть разные команды.
3. Остальные получают долю полного балла: какую часть «пути» лучшей команды вы прошли. Путь считается умножением: ускорение в 1,2 раза, сделанное дважды, — это 1,2 × 1,2 = 1,44 раза. Поэтому если у лучшей команды s = 1,44, а у вас s = 1,2, вы прошли половину пути и получаете половину баллов: 17,5 из 35.
4. Выигрыш меньше 5% (s или m меньше 1,05) не отличить от шума: он считается ничьей с базой и даёт 0 баллов. Разброс между повторами на T4 у нас 1–4%.

5. Потолка нет, ниже нуля баллы не опускаются.
6. При равной сумме баллов выше команда с лучшим результатом по экономии памяти.

Та же доля формулой (ln — натуральный логарифм):

$$\text{Скорость} = 35 \times \ln s / \ln s_{\text{лучш}}, \quad s = \text{время базы} / \text{ваше время}$$

$$\text{Память} = 30 \times \ln m / \ln m_{\text{лучш}}, \quad m = \text{пик памяти базы} / \text{ваш пик}$$

Посчитать самому в Excel или Google Таблицах: в ячейку A1 впишите своё s, в B1 — s лучшей команды, в C1 — формулу `=35*LN(A1)/LN(B1)`. Для памяти то же с m и формулой `=30*LN(A1)/LN(B1)`.

11.3 Сколько это в баллах

Скорость, если у лучшей команды s = 1,5:

Ваше s	1,04	1,05	1,1	1,2	1,3	1,4	1,5
Баллы	0	4,2	8,2	15,7	22,6	29,0	35

Память, если у лучшей команды m = 2:

Ваше m	1,04	1,05	1,1	1,25	1,5	1,75	2
Баллы	0	2,1	4,1	9,7	17,5	24,2	30

> **Пример итога одной команды**
>
> Решение обучается 64 с вместо 80 у базы (s = 1,25), а пик памяти у него в 1,5 раза меньше, чем у базы (m = 1,5). README жюри оценило в 12 баллов из 15. У лучших команд s = 1,5 и m = 2. Итог: 20 (работает) + 19,3 (скорость) + 17,5 (память) + 12 (README) = 68,8 балла из 100.

Допуск. Если не пройден хотя бы один пункт раздела 7, решение не допускается: баллов за скорость и память нет. Автоматические проверки предварительные: жюри читает код каждого допущенного решения, и его решение окончательное.

12. Сроки

Хакатон длится 36 часов, дедлайн сдачи — 31 октября в 23:59 по времени Астаны (18:59 UTC).

Дата	Событие
30 октября, 12:00	Старт, команды получают задачу и стартовый набор
31 октября, 23:59	Дедлайн сдачи на платформе: репозиторий (со ссылкой на Kaggle-ноутбук), видео, презентация

ТЕХНИКАЛЫҚ ТАПСЫРМА (ТТ)

Soup Kernel Challenge

---

1. Жалпы мәліметтер

Тапсырыс беруші	Soup
Хакатон	Decentrathon 6.0
Тапсырманың атауы	Soup Kernel Challenge
Тапсырма түрі	ML-инженерия, GPU-бағдарламалау (Triton), өнімділікті оңтайландыру
Формат	4 адамға дейінгі командалар, әр командадан бір нәтиже
Мерзімі	30 қазан 12:00 – 31 қазан 23:59, Астана уақытымен (36 сағат)
Бастапқы репозиторий	github.com/MakazhanAlpamys/soup-kernel-challenge (қазір жабық, 30 қазанда 12:00-де ашылады)

2. Мәселенің сипаттамасы

Тілдік модельдерді бір тегін бейнекартада қосымша оқыту модельге қолжетімділікке емес, жад пен уақытқа тіреледі. Қымбат GPU-ы жоқ студенттер, зерттеушілер мен әзірлеушілер Kaggle T4 сияқты карталарда жұмыс істейді (шамамен 15 ГБ, bf16 жоқ).

Тегін Kaggle T4-тегі біздің өлшемдер (Soup 0.77.0, Qwen2.5-1.5B, 4-бит, batch 4, seq 512, 256 қазақша нұсқаулық, 64 қадам):

- soup train командасы шамамен 80 секунд жүреді (оның шамамен бір минуты — оқытудың өзі, секундына шамамен 900 токен) және бейнекарта жадының 15 ГБ-ының шамамен 14 ГБ-ын алады;
- Unsloth сол деректер мен сол картада ұқсас LoRA-ны шамамен 2,3 ГБ жадпен оқытады, яғни 6 есе аз;
- Soup-тың осы тапсырмадағы бір қадамының профилі: GPU уақыттың шамамен 95%-ында бос емес; шамамен 36%-ы матрицаларды көбейтуге, шамамен 44%-ы ұсақ элементтік операциялар мен типтерді түрлендіруге, шамамен 7%-ы 152 мың токендік сөздік бойынша softmax-қа, шамамен 5%-ы attention-ға және шамамен 5%-ы 4-биттік салмақтарды ашуға кетеді.

Демек, қор бар, бірақ оны өз кернелдеріңізбен қаншалықты алға жылжытуға болатыны белгісіз. Хакатонда біз дәл осыны білгіміз келеді.

Құндылығы. Сол оқыту тегін картада тезірек жүреді және жадтан қор қалдырады: сол T4-те көбірек дерек, ұзынырақ мәтін немесе үлкенірек модель алуға болады.

3. Пайдаланушы және сценарий

Негізгі пайдаланушы — қымбат бейнекарталарға қолы жетпейтін, модельді тегін Kaggle T4-те қосымша оқытатын инженер немесе зерттеуші.

Негізгі сценарий:

1. Пайдаланушы тапсырма конфигімен `soup train` іске қосады.
2. Сіздің кернелдеріңіз Soup-қа бір apply() функциясымен қосылады (Unsloth та солай істейді), және сол іске қосу тезірек немесе азырақ жадпен жүреді.
3. Нәтижесінде сол сападағы сол адаптер шығады.

4. Хакатон тапсырмасы

Kaggle T4-тегі Soup-тың тіркелген оқыту іске қосылуын жеделдететін немесе сол сапада жад шығынын азайтатын өз Triton-кернелдеріңізді (және кез келген ілеспе өзгерістерді) жазыңыз.

Қай операцияларды жеделдету керегін және оны қалай істеу керегін өзіңіз шешесіз. Міндетті бір ғана шарт: шешімде оқытуда шынымен жұмыс істейтін кемінде бір өз Triton-кернеліңіз болуы керек. Нәтиже біздің тексерудегі өлшенген жылдамдық пен жад бойынша бағаланады, бұл шкалалардың шегі жоқ.

Бастапқы репозиторийде іске қосу мен өлшеуге керектің бәрі бар: тапсырма конфигі, деректерді жүктеу, кернелдің жұмыс істейтін мысалы және өлшеу скрипттері. Қазылар алқасы дәл осы скрипттермен тексереді, сондықтан нәтижеңізді тапсырмас бұрын көресіз.

5. Кіріс және шығыс

Кіріс. Soup 0.77.0 (pip install soup-cli[train]==0.77.0) және `task.yaml` тапсырма конфигі бар soup-kernel-challenge бастапқы репозиторийі:

- модель Qwen2.5-1.5B-Instruct, 4-биттік салмақтар (NF4), q_proj және v_proj үшін LoRA r=16, alpha=32;
- batch 4, ұзындығы 512 токенге дейін, 256 жол қазақша нұсқаулық, бір эпоха (64 қадам);
- орта: тегін Kaggle T4 (Python 3.13, torch 2.11, Triton 3.6.0 бар образ).

Қорытынды тексеруде оқыту мен бағалау сол жинақтың жасырын жолдарында және басқа seed-пен жүреді; қалған баптаулар өзгермейді.

Шығыс. Хакатон платформасы құрған, ішінде soup-kernel-challenge үлгісінің мазмұны бар команда репозиторийі (коммитке сілтеме): Triton-кернелдеріңіз бар solution/ пакеті және оларды Soup-қа қосатын apply() функциясы. Ол үлгідегі Kaggle-ноутбукта «бәрін орындау» командасымен өзгеріссіз іске қосылады. Біз soup train командасының толық уақытын (іске қосудан сақталған адаптерге дейін), GPU жадының шыңын және алынған адаптердің сапасын өлшейміз.

run_task.py шығаратын қорытындының мысалы (сандар шартты): `speedup 1.300x  memory base/solution 1.800x  eval loss vs base 0.1 %`. Сол сандар және әр іске қосу бойынша толық мәліметтер runs/result.json файлына сақталады.

6. Деректер

Тапсырмаға бір ашық жинақ жеткілікті, тіркелу мен рұқсат қажет емес.

Жинақ	AmanMussa/kazakh-instruction-v2 (Hugging Face), MIT лицензиясы: 52 201 қазақша нұсқаулық

Өрістер	`instruction`, `input`, `output` (alpaca форматы)
Сапасы	Stanford Alpaca жинағының Google Translate арқылы қазақшаға аудармасы, қателері қолмен түзетілген; Қазақстан тарихы мен мәдениеті туралы сұрақтар қосылған (авторы Mussa Aman, ҚазҰУ). Бос жауаптар сирек: 2000 жолға 2
Оқыту	алғашқы 256 жол
Өзін-өзі тексеруге арналған ашық іріктеме	40000–40255 жолдар
Жасырын іріктеме	сол жинақтың басқа жолдары, оларды ұйымдастырушылар таңдайды; қорытынды тексеруде оқыту мен бағалау осы жолдарда жүреді

Бір жол орта есеппен шамамен 220 токен алады (медианасы 140), жолдардың 6%-ы 512-ден ұзын және қысқартылады. Өз кернелдеріңізді тексеру үшін синтетикалық тензорларды қолдануға болады.

7. Must have

Шешім барлық бес тармақтан өтсе ғана орындалды деп саналады. 1–4 тармақтарды бастапқы репозиторийдегі скрипттер тексереді, оларды тапсырмас бұрын іске қосуға болады.

№	Талап	Қалай тексереміз
1	Шешім T4-тегі таза Kaggle-ноутбукта өзгеріссіз іске қосылады	біздің «бәрін орындау» іске қосуымыз
2	solution/ қалтасындағы өз Triton-кернелдеріңіз (`@triton.jit`) әрбір өлшенетін іске қосуда жұмыс істейді және бірге оқыту уақытының кемінде 5%-ын алады (3-қадамнан бастап саналады: алғашқы екеуі компиляцияға кетеді). Саналмайды: `torch.compile` кернелдері, мысалдағы кернелдер (қалай қайта жазылса да), көшірілген бөтен кернелдер және нәтижесі оқытуда пайдаланылмайтын кернелдер	кернелдер уақытын іске қосулардың өзінде өлшеу (CUDA-оқиғалар), бастапқы кодты талдау және қазылар алқасының кодты оқуы
3	Ешбір көрсеткіш бойынша базадан нашар емес: жасырын жолдардағы адаптер loss-ы базалық Soup-тан 0,5%-дан артық жоғары емес; уақыт пен жад шығыны базадан 10%-дан артық көп емес	eval_adapter.py (бәріне бір шығын функциясы) және run_task.py скрипттері

№	Талап	Қалай тексереміз
4	Сол тапсырма және регрессия жоқ: оқытудың сол баптаулары, сол адаптер және Soup шығыстары (қадамдар бойынша токен саны сәйкес келеді; rewind.jsonl-дағы энтропия, дәлдік және жолдар шығыны 3% шегінде); барлық қадамдардың шығын қисығы базаға жақын (бірінші қадам 1% шегінде, орта есеппен 3% шегінде); өлшеу таза (бір бейнекарта, іске қосу алдында бос, harness белгілеген аллокатор баптаулары); Soup тесттері (5 файлдағы 79 тест: config, formats, loader, trainer_init, callback) сіздің патчтарыңызбен өтеді	check_rules.py скрипті және pytest
5	README: не істелді, қалай іске қосу керек, қандай нәтижелер, Kaggle-ноутбукқа сілтеме	қазылар алқасының тексеруі

8. Қатаң шектеулер

Платформа. Тегін Kaggle T4 (екі картаның біреуі қолданылады), fp16, bf16 жоқ. Қорытынды тексеру тек осы платформада жүреді.

Қолжетімділік. Телефоны расталған өз Kaggle-аккаунтыңыз керек: онсыз Kaggle GPU мен интернет бермейді. Тегін квота — бір аккаунтқа аптасына шамамен 30 сағат GPU. Деректер ашық, тапсырмада дербес деректер жоқ.

Рұқсат етіледі. PyPI-дағы torch, triton, transformers, peft, trl, bitsandbytes пакеттері және soup-cli 0.77.0; ЖИ-агенттер; бөтен кернелдердің идеялары — дереккөзін міндетті түрде көрсетіп, бірақ олардың кодын емес.

Болмайды:

- дайын кернел-кітапханаларды қолдану (unsloth, liger-kernel, xformers, flash-attn, torchao және сол сияқтылар, толық тізім harness/common.py файлында), соның ішінде Soup-тың кірістірілген режимдерін: backend: unsloth және use_liger;
- `torch.compile` жасаған кернелдерді өзіңіздікі деп көрсету;
- тапсырманың өзін өзгерту: модельді, деректерді, батчтардың құрамы мен ретін, қадамдар санын, LoRA баптауларын (соның ішінде dropout), оптимизатор мен lr кестесін; Soup-тың кірістірілген режимдерін ауыстыру (gradient_checkpointing, activation_offloading, fast_lora, use_cut_ce, packing және сол сияқтылар); оқытылатын параметрлер қосу; Soup шығыстарын өшіру немесе олардың мәндерін өзгерту (логтағы метрикалар, rewind.jsonl). Батч ішіндегі pad-токендерді өз кодыңызбен алып тастауға болады, әртүрлі батчтардың жолдарын араластыруға болмайды;
- өлшеу және бағалау скрипттерін (harness/, tests/, task/) және олар белгілеген баптауларды, мысалы PyTorch жад аллокаторын өзгерту; жасырын деректерді іздеу немесе іріктеп табу;
- өлшеуді айналып өту: басқа бейнекарта, фондық процестер, өлшеу кезінде немесе тесттерде ерекше әрекет ету, сақталған адаптерді алмастыру, дайын компиляцияланған кэштер; жасырын жолдарды және өз репозиторийіңіз бен тапсырма деректерінен тыс кез келген файлдарды оқу;
- командадан бірден көп нәтиже тапсыру немесе 4 адамнан көп командамен қатысу;
- жабық код: тапсыру кезінде репозиторий мен Kaggle-ноутбук қазылар алқасының тексеруі үшін ашық болуы керек.

9. Ұсынылатын бағыттар

Бұл міндетті тізім емес, кеңестер: бағытты таңдау емес, өлшенген нәтиже бағаланады.

- **Ұсақ операцияларды біріктіру.** Soup-тың осы тапсырмадағы қадам профилі бойынша GPU уақытының шамамен 44%-ы элементтік операциялар мен типтерді түрлендіруге кетеді: fp32-дегі нормализациялар мен эмбеддингтер, LoRA, активациялар.
- **Біріктірілген шығын функциясы және үлкен сөздік.** 152 мың токендік сөздік бойынша логиттер гигабайттаған жадты және уақыттың шамамен 7%-ын алады; оларды loss-тан басқа энтропия мен дәлдікті есептеу және rewind журналы оқиды. Unsloth сол деректерде 6 есе аз жад алады.
- **4-биттік салмақтармен біріктірілген көбейту.** NF4-ті ашу қадам уақытының шамамен 5%-ын алады және жадта салмақтардың тығыз көшірмесін ұстайды; матрицаларды көбейтудің өзі тағы шамамен 36%-ын алады.
- **Активацияларды сығу, салмақтарды қайта орау, int8, батч ішіндегі pad-токендерді алып тастау.**

10. Тапсыру және артефактілер

Барлығы хакатон платформасында тапсырылады. Әр команда төрт нәрсе тапсырады:

1. **Командаңың GitHub-репозиторийі.** Барлық жұмыс хакатон платформасы командаға құрған GitHub-репозиторийде болуы керек: басқа репозиторийлердегі жұмысты тексермейміз. Онда solution/ пакеті, README (не істелді, қалай іске қосу керек, нәтижелер, шектеулер) және соңғы коммит пен Kaggle-ноутбукқа сілтемелері бар толтырылған SUBMISSION.md болады. Дедлайнға қарай репозиторий тексеру үшін ашық болуы керек: жалпыға ашық немесе платформа арқылы қазылар алқасына қолжетімді.
2. **Жалпыға ашық Kaggle-ноутбук** templates/kaggle_notebook.ipynb үлгісінен (нәтижелері бар сақталған нұсқа; ондағы COMMIT — соңғы коммит), ол «бәрін орындау» арқылы өзгеріссіз іске қосылады. Оған сілтеме репозиторийіңіздің README мен SUBMISSION.md файлдарында міндетті түрде болуы керек.
3. **Демо-бейне** 3 минутқа дейін: нені жеделдеттіңіз және не шықты.
4. **Презентация.**

Презентацияда не көрсету керек:

- қай операцияны жеделдеттіңіз және неге;
- кернел қалай құрылған;
- базамен салыстырғандағы жылдамдық, жад және сапа нәтижелері;
- шешімнің шектеулері және келесі қадамдар.

10.1 Үлгіні платформа берген репозиторийге қалай көшіру керек

«Use this template» батырмасы жаңа репозиторий құрады және үлгіні бұрыннан бар репозиторийге көшірмейді. Платформа берген репозиторий бос болса немесе онда тек README болса, оны клондаңыз (`git clone`) және оның қалтасында мына командаларды орындаңыз:

```
git fetch https://github.com/MakazhanAlpamys/soup-kernel-challenge.git main
git checkout FETCH_HEAD -- .
git commit -m "Soup Kernel Challenge starter kit"
git push origin HEAD
```

Үлгі файлдары аттас файлдарды ауыстырады (мысалы, README.md), репозиторийдегі қалғаны сақталады. Git-сіз: үлгіні жүктеп алыңыз (Code → Download ZIP), архивтен шығарып, файлдарды өз репозиторийіңізге жүктеңіз. Егер платформа репозиторийді үлгімен бірге құрған болса, ештеңе көшірудің қажеті жоқ.

11. Бағалау критерийлері

Қорытындыны Kaggle T4-тегі өз іске қосуымызбен есептейміз: бастапқы репозиторийдегі сол скрипттер, бірақ деректердің жасырын жолдары. Базалық Soup пен сіздің шешіміңізді бір сессияда екі реттен іске қосамыз (база, шешім, шешім, база ретімен) және әр тарап үшін екі іске қосудың орташасын аламыз. Кернелдерді компиляциялау уақыты өлшемге кіреді.

Критерий	Нені бағалаймыз	Ұпай
Жұмыс істейді және дұрыс	Шешім 7-бөлімнің барлық бес тармағынан өтеді. Бәрі немесе ештеңе: барлық тармақтан өтсе — 20 ұпай. Кемінде біреуінен өтпесе, шешім жіберілмейді және рейтингке кірмейді.	20
Жылдамдық	`soup train` командасының толық уақыты: іске қосудан сақталған адаптерге дейін, модельді жүктеу мен кернелдерді компиляциялауды қоса алғанда. s = база уақыты / сіздің уақытыңыз. Жіберілген командалар арасындағы ең үлкен s 35 ұпай алады, қалғандары азырақ (қалай — төменде).	35
Жад	Бүкіл іске қосу бойынша бейнекарта жадының шыңы (nvidia-smi, әр 0,1 с сайын өлшенеді). m = база шыңы / сіздің шыңыңыз. Жіберілген командалар арасындағы ең үлкен m 30 ұпай алады, қалғандары азырақ.	30
README және нәтижені қайталау	Қазылар алқасы үш бөлік бойынша бағалайды, әрқайсысына 5 ұпайға дейін (11.1-бөлім).	15
Барлығы		**100**

11.1 README: 15 ұпай неден құралады

Бөлік	5 ұпай, егер	Ұпай
Не істелді	қай операцияларды неге жеделдеткеніңіз, кернелдеріңіз қалай құрылғаны және идеялар қайдан алынғаны (дереккөздерімен) түсінікті	5-ке дейін
Қалай іске қосу керек	қазылар алқасы нәтижеңізді командаға сұрақ қоймай, README бойынша қайталай алады: іске қосу қадамдары, Kaggle-ноутбук пен соңғы коммитке сілтемелер	5-ке дейін
Нәтижелер және шектеулер	өз іске қосуыңыздан алынған базамен салыстыру кестесі (уақыт, жад, loss) және не шықпағаны мен шешімнің қай жерде әлсіз екені адал айтылған	5-ке дейін

Толық орындалмаған бөлік 1-ден 4-ке дейін ұпай алады, жоқ бөлік — 0.

11.2 Жылдамдық пен жад үшін ұпай қарапайым тілмен

1. Өз s пен m мәндеріңізді біліңіз. Оларды run_task.py ноутбуктің соңында шығарады: `speedup` — бұл s, `memory base/solution` — бұл m. Мысалы, база 80 с оқытылады, сіздің шешіміңіз 64 с: s = 80 / 64 = 1,25.
2. Үздік команда толық ұпай алады: жіберілген командалар арасындағы ең үлкен s — жылдамдық үшін 35 ұпай, ең үлкен m — жад үшін 30 ұпай. Жылдамдық пен жад бойынша үздіктер әртүрлі командалар болуы мүмкін.
3. Қалғандары толық ұпайдың үлесін алады: үздік команда жүріп өткен «жолдың» қанша бөлігін өткеніңізге қарай. Жол көбейту арқылы есептеледі: екі рет жасалған 1,2 есе жеделдету — бұл 1,2 × 1,2 = 1,44 есе. Сондықтан үздік командада s = 1,44, ал сізде s = 1,2 болса, сіз жолдың жартысын өттіңіз және ұпайдың жартысын аласыз: 35-тен 17,5.
4. 5%-дан аз ұтыс (s немесе m 1,05-тен кіші) шуылдан ажыратылмайды: ол базамен тең нәтиже деп саналады және 0 ұпай береді. Біздің өлшемдерде T4-тегі қайталаулар арасындағы шашырау 1–4%.
5. Жоғарғы шегі жоқ, ұпай нөлден төмен түспейді.
6. Ұпай қосындысы тең болса, жадты үнемдеу нәтижесі жақсырақ команда жоғары тұрады.

Сол үлестің формуласы (ln — натурал логарифм):

$$\text{Жылдамдық} = 35 \times \ln s / \ln s_{\text{үздік}}, \quad s = \text{база уақыты / сіздің уақытыңыз}$$

$$\text{Жад} = 30 \times \ln m / \ln m_{\text{үздік}}, \quad m = \text{база жадының шыңы / сіздің шыңыңыз}$$

Excel немесе Google Sheets-те өзіңіз есептеу үшін: A1 ұяшығына өз s мәніңізді, B1 ұяшығына үздік команданың s мәнін, C1 ұяшығына `=35*LN(A1)/LN(B1)` формуласын жазыңыз. Жад үшін де солай, m мәндерімен және `=30*LN(A1)/LN(B1)` формуласымен.

11.3 Бұл қанша ұпай болады

Жылдамдық, үздік командада s = 1,5 болса:

Сіздің s	1,04	1,05	1,1	1,2	1,3	1,4	1,5
Ұпай	0	4,2	8,2	15,7	22,6	29,0	35

Жад, үздік командада m = 2 болса:

Сіздің m	1,04	1,05	1,1	1,25	1,5	1,75	2
Ұпай	0	2,1	4,1	9,7	17,5	24,2	30

> **Бір командаңың қорытынды мысалы**
>
> Шешім базадағы 80 с орнына 64 с оқытылады (s = 1,25), ал оның жад шыңы базадан 1,5 есе аз (m = 1,5). Қазылар алқасы README-ді 15 ұпайдың 12-сіне бағалады. Үздік командаларда s = 1,5 және m = 2. Қорытынды: 20 (жұмыс істейді) + 19,3 (жылдамдық) + 17,5 (жад) + 12 (README) = 100 ұпайдың 68,8-і.

Жіберілу шарты. Егер 7-бөлімнің кемінде бір тармағынан өтпесе, шешім жіберілмейді: жылдамдық пен жад үшін ұпай берілмейді. Автоматты тексерулер алдын ала жүреді: қазылар алқасы әрбір жіберілген шешімнің кодын оқиды, оның шешімі түпкілікті.

12. Мерзімдер

Хакатон 36 сағатқа созылады, тапсыру дедлайны — 31 қазан, 23:59, Астана уақытымен (18:59 UTC).

Күні	**Оқиға**
30 қазан, 12:00	Басталуы: командалар тапсырма мен бастапқы жинақты алады
31 қазан, 23:59	Платформада тапсыру дедлайны: репозиторий (Kaggle-ноутбукқа сілтемесімен), бейне, презентация

TECHNICAL SPECIFICATION

Soup Kernel Challenge

---

1. General information

Client	Soup
Hackathon	Decentrathon 6.0
Case name	Soup Kernel Challenge
Case type	ML engineering, GPU programming (Triton), performance optimization
Format	teams of up to 4 people, one result per team
Dates	30 October 12:00 – 31 October 23:59, Astana time (36 hours)
Starter repository	github.com/MakazhanAlpamys/soup-kernel-challenge (private now, opens on 30 October at 12:00)

2. Problem

Fine-tuning language models on a single free GPU is limited by memory and time, not by access to the model. Students, researchers and developers without expensive GPUs work on cards such as the Kaggle T4 (about 15 GB, no bf16).

Our measurements on a free Kaggle T4 (Soup 0.77.0, Qwen2.5-1.5B, 4-bit, batch 4, seq 512, 256 Kazakh instructions, 64 steps):

- the soup train command takes about 80 seconds (about a minute of it is the training itself, about 900 tokens/s) and uses about 14 of the 15 GB of GPU memory;
- Unsloth trains a similar LoRA on the same data and the same GPU in about 2.3 GB, that is 6 times less;
- the step profile of Soup itself on this task: the GPU is busy about 95% of the time; about 36% goes to matrix multiplication, about 44% to small element-wise operations and dtype casts, about 7% to the softmax over the 152 thousand token vocabulary, about 5% to attention and about 5% to unpacking the 4-bit weights.

So there is headroom, but it is unclear how far it can be pushed with your own kernels. That is what we want to find out at the hackathon.

Value. The same training on a free GPU runs faster and leaves spare memory: on the same T4 you can take more data, longer texts or a larger model.

3. User and scenario

The main user is an engineer or researcher without access to expensive GPUs who fine-tunes a model on a free Kaggle T4.

Key scenario:

1. The user runs `soup train` with the task config.
2. Your kernels plug into Soup through a single apply() function (as Unsloth does), and the same run is faster or uses less memory.
3. The output is the same adapter of the same quality.

4. Hackathon task

Write your own Triton kernels (and any supporting changes) that speed up a fixed Soup training run on a Kaggle T4 or cut its memory use at the same quality.

Which operations to speed up and how is up to you. One thing is required: the solution has at least one Triton kernel of your own that really runs in training. The result is scored by the speed and memory measured in our check, and these scales have no ceiling.

The starter repository has everything for running and measuring: the task config, data loading, a working example kernel and the measurement scripts. These are the same scripts the jury uses, so you see your result before you submit.

5. Input and output

Input. Soup 0.77.0 (pip install soup-cli[train]==0.77.0) and the starter repository soup-kernel-challenge with the task config `task.yaml`:

- model Qwen2.5-1.5B-Instruct, 4-bit weights (NF4), LoRA r=16, alpha=32 on q_proj and v_proj;
- batch 4, length up to 512 tokens, 256 rows of Kazakh instructions, one epoch (64 steps);
- environment: a free Kaggle T4 (image with Python 3.13, torch 2.11, Triton 3.6.0).

In the final check, training and evaluation run on hidden rows of the same dataset and with a different seed; all other settings stay the same.

Output. The team repository created by the hackathon platform, with the contents of the template soup-kernel-challenge (link to a commit): a solution/ package with your Triton kernels and an apply() function that plugs them into Soup. It runs in the Kaggle notebook from the template with “Run All”, without edits. We measure the full time of the soup train command (from start to the saved adapter), the peak GPU memory and the quality of the resulting adapter.

Example of the summary printed by run_task.py (made-up numbers): `speedup 1.300x  memory base/solution 1.800x  eval loss vs base 0.1 %`. The same numbers and details of every run are saved to runs/result.json.

6. Data

One open dataset is enough for the task; no registration or access request is needed.

Dataset	AmanMussa/kazakh-instruction-v2 (Hugging Face), MIT license: 52,201 Kazakh instructions
Fields	`instruction`, `input`, `output` (alpaca format)

Quality	Stanford Alpaca translated into Kazakh with Google Translate, errors fixed by hand; questions on the history and culture of Kazakhstan added (author Mussa Aman, KazNU). Empty answers are rare: 2 per 2,000 rows
Training	the first 256 rows
Open rows for self-checks	rows 40000–40255
Hidden rows	other rows of the same dataset, chosen by the organizers; the final check trains and evaluates on them

A row is about 220 tokens on average (median 140); 6% of rows are longer than 512 and are truncated. Synthetic tensors are fine for testing your kernels.

7. Must have

A solution counts as done only if it passes all five items. Items 1–4 are checked by the scripts in the starter repository, so you can run them before submitting.

No.	Requirement	How we check
1	The solution runs in a clean Kaggle notebook on a T4 without edits	our “Run All” run
2	Your own Triton kernels (`@triton.jit`) from the solution/ folder run in every measured run and together take at least 5% of the training time (counted from step 3: the first two steps go to compilation). Not counted: kernels from `torch.compile`, the example's kernels (however rewritten), copied kernels of others and kernels whose result is not used in training	kernel timing inside the measured runs (CUDA events), source analysis and code reading by the jury
3	No worse than the base on any metric: the adapter's loss on the hidden rows is at most 0.5% above the base Soup; time and peak memory are at most 10% above the base	the eval_adapter.py script (one loss function for everyone) and run_task.py
4	The same task and no regressions: the same training settings, adapter and Soup outputs (token counts per step match; entropy, accuracy and per-row losses in rewind.jsonl within 3%); the loss curve of all steps is close to the base (first step within 1%, on average within 3%); a clean measurement (one GPU, idle before the run, the harness's allocator settings); the Soup tests (79 tests in 5 files: config, formats, loader, trainer_init, callback) pass with your patches	the check_rules.py script and pytest
5	README: what was done, how to run it, what the results are, a link to the Kaggle notebook	jury review

8. Hard constraints

Platform. A free Kaggle T4 (one of the two cards is used), fp16, no bf16. The final check runs on this platform only.

Access. You need your own Kaggle account with a verified phone number: without it Kaggle gives no GPU and no internet. The free quota is about 30 GPU hours a week per account. The data is open; the task has no personal data.

Allowed. The packages torch, triton, transformers, peft, trl, bitsandbytes and soup-cli 0.77.0 from PyPI; AI agents; ideas from other people's kernels with the source always credited, but not their code.

Not allowed:

- using ready-made kernel libraries (unsloth, liger-kernel, xformers, flash-attn, torchao and the like, full list in harness/common.py), including Soup's built-in modes backend: unsloth and use_liger;
- passing off kernels generated by `torch.compile` as your own;
- changing the task itself: the model, the data, the content and order of batches, the number of steps, the LoRA settings (dropout included), the optimizer and the lr schedule; switching Soup's built-in modes (gradient_checkpointing, activation_offloading, fast_lora, use_cut_ce, packing and the like); adding trainable parameters; turning off Soup's outputs or changing their values (metrics in the log, rewind.jsonl). Removing pad tokens inside a batch with your own code is allowed; mixing rows of different batches is not;
- changing the measurement and evaluation scripts (harness/, tests/, task/) or the settings they set, such as the PyTorch memory allocator; searching for or guessing the hidden data;
- getting around the measurement: another GPU, background processes, special behavior under measurement or in tests, swapping the saved adapter, ready-made compiled caches; reading the hidden rows or any files outside your repository and the task data;
- submitting more than one result per team, or a team of more than 4 people;
- closed code: by submission time the repository and the Kaggle notebook must be open for the jury to check.

9. Suggested directions

These are hints, not a required list: the measured result is scored, not the choice of direction.

- **Fusing small operations.** By our step profile of Soup on this task, about 44% of GPU time goes to element-wise operations and dtype casts: normalizations and embeddings in fp32, LoRA, activations.
- **Fused loss and the large vocabulary.** The logits over the 152 thousand token vocabulary take gigabytes of memory and about 7% of the time; besides the loss, the entropy and accuracy metrics and the rewind log read them. Unsloth uses 6 times less memory on the same data.
- **Fused matrix multiplication with 4-bit weights.** Unpacking NF4 takes about 5% of the step time and keeps a dense copy of the weights in memory; the matrix multiplication itself takes another 36% or so.
- **Activation compression, weight repacking, int8, removing pad tokens inside a batch.**

10. Submission and deliverables

Everything is submitted on the hackathon platform. Each team submits four things:

1. **The team's GitHub repository.** All work must be in the GitHub repository that the hackathon platform created for the team: we do not check work in other repositories. It holds the solution/ package, a README (what was done, how to run it, results, limitations) and a filled-in SUBMISSION.md with links to the final commit and to the Kaggle notebook. By the deadline the repository must be open for checking: public, or with access for the jury through the platform.
2. **A public Kaggle notebook** from the template templates/kaggle_notebook.ipynb (a saved version with results; its COMMIT is the final commit) that runs with “Run All” without edits. A link to it must be in the README and in the SUBMISSION.md of your repository.
3. **A demo video** of up to 3 minutes: what you sped up and what came out of it.
4. **A presentation.**

What the presentation should cover:

- which operation you sped up and why;
- how the kernel works;
- results on speed, memory and quality against the base;
- limitations of the solution and next steps.

10.1 How to move the template into the platform's repository

The “Use this template” button creates a new repository and does not copy the template into an existing one. If the repository from the platform is empty or holds only a README, clone it (`git clone`) and run this in its folder:

```
git fetch https://github.com/MakazhanAlpamys/soup-kernel-challenge.git main
git checkout FETCH_HEAD -- .
git commit -m "Soup Kernel Challenge starter kit"
git push origin HEAD
```

The template's files replace files with the same names (for example, README.md); everything else in the repository stays. Without git: download the template (Code → Download ZIP), unpack it and upload the files to your repository. If the platform created the repository with the template already inside, there is nothing to move.

11. Evaluation criteria

The result is our own run on a Kaggle T4: the same scripts as in the starter repository, but the hidden data rows. We run the base Soup and your solution twice each in one session, in the order base, solution, solution, base, and take the average of the two runs for each side. Kernel compilation time is part of the measurement.

Criterion	What we score	Points
Works and correct	The solution passes all five items of section 7. All or nothing: 20 points if every item passes. If even one fails, the solution is not admitted and is not ranked.	20

Criterion	What we score	Points
Speed	The full time of the `soup train` command: from start to the saved adapter, including model loading and kernel compilation. s = base time / your time. The largest s among admitted teams gets 35 points, the others get less (how exactly is below).	35
Memory	The GPU memory peak over the whole run (nvidia-smi, sampled every 0.1 s). m = base peak / your peak. The largest m among admitted teams gets 30 points, the others get less.	30
README and reproducibility	Scored by the jury in three parts, up to 5 points each (section 11.1).	15
Total		**100**

11.1 README: what the 15 points are made of

Part	5 points if	Points
What was done	it is clear which operations you sped up and why, how your kernels work and where the ideas came from (with sources)	up to 5
How to run it	the jury reproduces your result from the README without asking the team: run steps, links to the Kaggle notebook and the final commit	up to 5
Results and limitations	a table against the base from your run (time, memory, loss) and an honest account of what did not work and where the solution is weaker	up to 5

A part done only partly gets 1 to 4 points; a missing part gets 0.

11.2 Speed and memory points in plain words

1. Find your s and m. run_task.py prints them at the end of the notebook: `speedup` is s, `memory base/solution` is m. For example, the base trains in 80 s and your solution in 64 s: s = 80 / 64 = 1.25.
2. The best team gets the full points: the largest s among admitted teams gets 35 points for speed, the largest m gets 30 points for memory. The best team on speed and the best on memory can be different teams.
3. The others get a share of the full points: the part of the best team's “way” that you covered. The way is counted by multiplying: a 1.2× speedup done twice is 1.2 × 1.2 = 1.44×. So if the best team has s = 1.44 and you have s = 1.2, you covered half the way and get half the points: 17.5 of 35.
4. A gain under 5% (s or m below 1.05) cannot be told apart from noise: it counts as a tie with the base and gives 0 points. The spread between repeats on the T4 is 1–4% in our runs.
5. There is no ceiling, and points never go below zero.
6. If teams have the same total, the team with the better memory result ranks higher.

The same share as a formula (ln is the natural logarithm):

$$\text{Speed} = 35 \times \ln s / \ln s_{best}, \quad s = \text{base time} / \text{your time}$$

$$\text{Memory} = 30 \times \ln m / \ln m_{best}, \quad m = \text{base memory peak} / \text{your peak}$$

To work it out yourself in Excel or Google Sheets: put your s in cell A1, the best team's s in B1 and the formula `=35*LN(A1)/LN(B1)` in C1. For memory do the same with m and the formula `=30*LN(A1)/LN(B1)`.

11.3 What it means in points

Speed, if the best team has s = 1.5:

Your s	1.04	1.05	1.1	1.2	1.3	1.4	1.5
Points	0	4.2	8.2	15.7	22.6	29.0	35

Memory, if the best team has m = 2:

Your m	1.04	1.05	1.1	1.25	1.5	1.75	2
Points	0	2.1	4.1	9.7	17.5	24.2	30

> **Example total for one team**
>
> The solution trains in 64 s instead of the base's 80 (s = 1.25), and its memory peak is 1.5 times lower than the base's (m = 1.5). The jury gave the README 12 points of 15. The best teams have s = 1.5 and m = 2. Total: 20 (works) + 19.3 (speed) + 17.5 (memory) + 12 (README) = 68.8 points of 100.

Admission. If even one item of section 7 fails, the solution is not admitted: it gets no points for speed or memory. The automatic checks are preliminary: the jury reads the code of every admitted solution, and the jury's decision is final.

12. Timeline

The hackathon lasts 36 hours; the submission deadline is 31 October at 23:59 Astana time (18:59 UTC).

Date	Event
30 October, 12:00	Start: teams get the task and the starter kit
31 October, 23:59	Submission deadline on the platform: repository (with the link to the Kaggle notebook), video, presentation